| **Nombres (EQUIPO 3):** | **Matrícula**: |
|---|---|
| Cristopher Alan Pulgarin Santa Cruz | 2105894 |
| Francisco Sebastián Hernández Terán | 2115798 |
| Jesús Orlando Moreno Chávez | 2108300 |

| **Nombre del curso:** Componentes de Lógica de Negocios | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
|---|---|
| **Modulo Netacad**: 1 | **Actividad**: UNIDAD 3 - DISEÑO ARQUITECTÓNICO DEL SISTEMA |

**Fecha**: 16/10/2026

**Bibliografía**


## Registros de Decisiones de Arquitectura (ADR)

- Richards, M. y Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

## Diagramas UML (Clases y Secuencia)

Para justificar la notación y el modelado visual del Anexo A y el Capítulo 8:

- Booch, G., Rumbaugh, J. y Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Addison-Wesley.

# Sazon Urbano - Architecture Decision Records (ADR)

## ADR-001: Selección del patrón de lógica de negocio: Domain Model

| Campo | Contenido |
|---|---|
| **Estado** | Aceptada (15 de octubre de 2026). |
| **Contexto** | Sazón Urbano requiere gestionar la recepción de pedidos por diversos canales, el control de existencias de ingredientes, modificaciones, cancelaciones y actualizaciones de estado de preparación. El caso presenta reglas de negocio que interactúan fuertemente con el ciclo de vida del pedido. |
| **Alternativas** | 1. **Transaction Script:** Simple, pero las reglas de inventario crítico y validación de modificaciones terminarían acopladas en funciones gigantes.<br><br>2. **Table Module:** Útil para tablas estáticas, pero Sazón Urbano tiene dependencias fuertes que atraviesan varias tablas.<br><br>3. **Domain Model:** Encapsula el comportamiento y estados cerca de las entidades de negocio. |
| **Decisión** | Se adopta Domain Model como patrón principal. La entidad Pedido controlará su ciclo de vida y sus invariantes (modificaciones, estados). Las validaciones operativas puras se delegan a las entidades en memoria antes de la persistencia. |
| **Consecuencias** | **Positivas:** Lógica centralizada de estados del pedido; facilita el cumplimiento de la trazabilidad requerida en modificaciones.<br><br>**Costos:** Mayor curva de aprendizaje, requiere definir muy bien los Aggregate Roots y crear más clases e interfaces para separar el dominio de la persistencia. |
| **Trazabilidad** | NB-01, NB-02, NB-06; US-02, US-04, US-05, US-06, US-08, US-09, US-10. |

## 9.2 ADR-002: Consistencia transaccional eventual y Pedido como Aggregate Root

| Campo | Contenido |
|---|---|
| **Estado** | Aceptada (15 de octubre de 2026). |
| **Contexto** | El proceso central del negocio gira alrededor del pedido. Un pedido agrupa productos y modificaciones, y al concretar una entrega, el inventario debe descontarse con precisión. |
| **Alternativas** | 1. **Inventario como raíz global:** Evita sobreventas, pero causaría problemas de concurrencia e impacto de rendimiento masivo.<br><br>2. **Producto como centro:** Incompatible con el ciclo de vida real que pertenece al pedido completo.<br><br>3. **Pedido como Aggregate Root (Consistencia Fuerte):** Bloquea las tablas durante el descuento.<br><br>4. **Pedido e Inventario separados (Consistencia Eventual):** Elegida para maximizar el rendimiento. |
| **Decisión** | La entidad Pedido será un Aggregate Root independiente que manejará su transición de estado ('Registrado', 'Entregado', etc.). Para evitar cuellos de botella, **se rechaza el descuento en una misma transacción**. El *Unit of Work* confirmará la entrega del pedido y despachará el evento PedidoEntregadoEvent. Un manejador asíncrono actualizará el Inventario en una transacción completamente separada. |
| **Consecuencias** | El límite de consistencia del Pedido queda claro, evitando bloquear todo el sistema cuando se preparan diferentes pedidos. Las reglas de modificaciones o cancelaciones se validan únicamente dentro del agregado Pedido. Existe un mínimo desfase de milisegundos en la vista del inventario. |
| **Trazabilidad** | NB-04, NB-05, NB-07; US-06, US-07, US-10, US-12. |

## 9.3 ADR-003: CQRS básico y Application Services delgados

| Campo | Contenido |
|---|---|
| **Estado** | Aceptada (15 de octubre de 2026). |
| **Contexto** | Sazón Urbano tiene dos necesidades distintas: alta transaccionalidad de escritura por parte de meseros y cocina (crear pedidos, actualizar estados), y por el otro, consultas de historial y cruces de ventas por parte del gerente y encargado. |
| **Alternativas** | 1. **Arquitectura tradicional N-Capas:** Acopla los modelos de lectura y escritura, haciendo las consultas lentas al cargar entidades pesadas.<br><br>2. **CQRS Básico:** Elegida para aislar las intenciones de lectura y escritura. |
| **Decisión** | Aplicar un patrón de CQRS básico: *Commands* para acciones de escritura (ej. RegistrarPedidoCommand) y *Queries* para lectura (ej. ConsultarDisponibilidadQuery). Los *Application Services* serán delgados: solo recibirán el comando, cargarán el agregado, ejecutarán el método de dominio y cerrarán la transacción con el *UnitOfWork*. Las lecturas devolverán directamente *ViewModels*. |
| **Consecuencias** | **Positivas:** Permite que los reportes de ventas e inventario que requiere el gerente se procesen rápidamente sin interferir con las operaciones de la caja y los meseros.<br><br>**Costos:** Se requiere implementar un número mayor de Handlers y DTOs para separar la lectura de la escritura. |
| **Trazabilidad** | NB-03, NB-09; US-01, US-11, US-13, US-14, US-15. |

## ADR-004: Control de acceso (RBAC) mediante interceptores

| Campo | Contenido |
|---|---|
| **Estado** | Aceptada (15 de octubre de 2026). |
| **Contexto** | Sazón Urbano requiere restringir ciertas operaciones críticas en el sistema. Específicamente, el caso dicta que solo el Encargado o Gerente tiene la facultad de autorizar la cancelación de un pedido, evitando que un Mesero lo haga unilateralmente. |
| **Alternativas** | 1. **Validación en la Capa de Dominio:** Descartada porque obligaría a inyectar datos de sesión (usuario, rol) en las entidades, mezclando seguridad con reglas comerciales.<br>2. **Validación en la Service Layer (Elegida):** Utilizar un middleware o interceptor antes de la ejecución del comando. |
| **Decisión** | El control de acceso basado en roles (RBAC) se manejará a través de un interceptor en la capa de aplicación. Antes de que se ejecute el CancelarPedidoHandler, el interceptor validará el rol del usuario. La capa de Dominio permanecerá totalmente agnóstica a las sesiones de los usuarios. |
| **Consecuencias** | **Positivas:** Respeta el Principio de Responsabilidad Única (SRP) y mantiene el modelo de dominio limpio de dependencias de infraestructura.<br>**Costos:** Requiere que la identidad del usuario (userId, rol) se extraiga en la capa de aplicación y viaje como parámetro hasta los eventos de dominio para poder guardarse en la auditoría. |
| **Trazabilidad** | US-10 (Registrar cancelación autorizada por encargado). |

## ADR-005: Historial inmutable de auditoría (Append-Only)

| Campo | Contenido |
|---|---|
| **Estado** | Aceptada (15 de octubre de 2026). |
| **Contexto** | Uno de los tres problemas más urgentes del restaurante es la "Dificultad para rastrear modificaciones y cancelaciones". Se requiere una trazabilidad absoluta donde el 100% de las acciones queden registradas con fecha, hora, usuario y motivo para auditoría. |
| **Alternativas** | 1. **Sobrescribir registros (Update):** Descartada, ya que se pierde el historial de los cambios anteriores.<br>2. **Event Sourcing:** Descartada por ser una sobreingeniería excesiva para el alcance de este proyecto escolar.<br>3. **Tablas Append-only relacionales (Elegida):** Un punto intermedio efectivo y simple. |
| **Decisión** | Se implementará un LogAuditoria en una tabla de solo anexado (*Append-Only*). Las operaciones que muten el estado (como modificaciones o cancelaciones) emitirán eventos de dominio. Un *Event Handler* asíncrono capturará estos eventos e insertará un nuevo registro en la tabla de auditoría, sin modificar ni borrar jamás los registros previos. |
| **Consecuencias** | **Positivas:** Resuelve de tajo la urgencia de trazabilidad del negocio y proporciona datos precisos para que el gerente detecte anomalías (NB-09).<br>**Costos:** Aumenta ligeramente el uso de almacenamiento en la base de datos a largo plazo. |
| **Trazabilidad** | NB-04, NB-05, NB-09; US-09, US-10, US-11, US-13. |